

Asistente Turístico Inteligente

RutaCamba — Reconocimiento de Lugares, Verificación de Identidad
y Traducción Multilingüe

Informe de Proyecto Final
Redes Neuronales y Aprendizaje Profundo

Universidad Católica Boliviana “San Pablo” — Sede Santa Cruz

Equipo: Diego Lewenstein (Datos) · Leandro Miranda (Re-ID, embeddings) · Jose Alfredo (Re-ID, LLM,
entrenamiento B2/B3/BA) · Alejandro Ojeda (Landmarks, B1/B2) · Nicole Lozada (WandB, API/UI,
informe)

Repositorio: github.com/leandromiranda-dev/RutaCamba · WandB:
wandb.ai/lozadaleonn-ciencialink/rutacamba

Santa Cruz de la Sierra, junio de 2026

1. Contexto y objetivos del proyecto

P: ¿Qué problema resuelve el sistema y para quién?

R: RutaCamba es el sistema técnico que responde al enunciado de negocio de “RutaCruceña”: una app de turismo para guías acreditados de Santa Cruz de la Sierra. El guía apunta la cámara a un lugar emblemático y el sistema responde con el nombre del sitio, una descripción y traducciones, sin depender de GPS. Por ser un servicio de pago, cada solicitud de reconocimiento se condiciona a una verificación previa de identidad (re-identificación biométrica), evitando llamadas no autorizadas al modelo de visión.

P: ¿Cuál es el flujo end-to-end exigido y cómo se mapea a los módulos del sistema?

R: (1) El usuario declara una identidad y sube una selfie → **Módulo A (Re-ID)** decide si concede acceso comparando el embedding contra la galería de guías registrados bajo un umbral de distancia. (2) Con acceso concedido, sube la foto de un lugar → **Módulo B** predice el landmark (top-k + probabilidades) con dos enfoques entrenados sobre el mismo dataset: B1 (CNN desde cero) y B2 (arquitectura con bloques residuales, nombrada BA en el código, comparada frente a B1). (3) La respuesta se traduce a inglés, francés e italiano → **Módulo C (LLM)**, con fallback offline si el LLM falla. (4) Todo el entrenamiento quedó registrado en **Módulo D (WandB)**. (5) Todo se sirve mediante **Módulo E**: una API FastAPI y una interfaz web (React) que la consume.

P: ¿Qué arquitectura general de software se usó?

R: Backend: FastAPI + PyTorch (TorchScript para inferencia) + facenet-pytorch (FaceNet/InceptionResnetV1, Re-ID) + LLM vía OpenRouter/Gemini con fallback NLLB-200. Frontend: React + TypeScript + Vite + Tailwind, con soporte PWA. Tracking: Weights & Biases (proyecto lozadaleonn-ciencialink/rutacamba). El repositorio organiza el código en src/data, src/landmarks, src/reid, src/translation, api/, web/, ETL/, notebooks/, docs/decisiones/, siguiendo la estructura sugerida por el enunciado.

Nota de alcance sobre la interfaz

R: El enunciado permite Gradio o Streamlit como interfaz mínima; el equipo optó por construir además una interfaz web propia en React consumiendo la misma API, documentada en web/README.md y servida en el puerto 5173 durante desarrollo.

2. Módulo A — Verificación de identidad por Re-ID

P: ¿Qué modelo de embeddings se usó y por qué, en lugar de ArcFace o VGG-Face?

R: Se usó **FaceNet** —en concreto InceptionResnetV1 preentrenado en **VGGFace2**, vía la librería `facenet-pytorch` (detección del rostro con MTCNN + extracción del embedding de 512-d). FaceNet entrena con *Triplet Loss*, que optimiza directamente el espacio de embeddings para que las imágenes de una misma identidad queden cercanas y las de identidades distintas queden separadas por un margen, produciendo vectores compactos y L2-normalizados robustos a cambios de iluminación y pose. Se eligió FaceNet por integrarse de forma nativa con el stack PyTorch del resto del proyecto —sin sumar la dependencia pesada de DeepFace/TensorFlow— y por ofrecer un pipeline MTCNN+InceptionResnetV1 listo y bien mantenido. Se descartó VGG-Face por ser una arquitectura más antigua con embeddings menos discriminativos para clusters complejos.

P: ¿Cómo se agregan varias fotos de referencia por identidad en la galería?

R: Por **distancia mínima** (best-match): la identidad queda representada por su foto más cercana al probe. Se descartó el promedio porque es más sensible a fotos borrosas o con mal encuadre, y el centroide porque con solo 3–5 fotos por identidad es inestable.

P: ¿Qué métrica de distancia se usó y por qué coseno y no euclidiana?

R: Distancia coseno ($1 - \cos(a,b)$). Los embeddings de FaceNet están L2-normalizados sobre la hiperesfera unitaria, de modo que la *dirección* del vector concentra toda la información discriminativa y su magnitud no aporta señal; la distancia coseno es la métrica natural en ese espacio, mientras que la euclidiana introduciría ruido de escala. Con FaceNet+coseno, los pares genuinos (misma persona) quedan a una distancia notablemente menor que los impostores, dando una separación clara para fijar el umbral.

P: ¿Cómo se justificó el umbral de acceso?

R: El umbral de partida es 0.65 (`config.REID_THRESHOLD`), tomado como punto de partida razonable para FaceNet+coseno y pensado para ajustarse formalmente. El método formal para calibrarlo es ROC/EER: se generan distribuciones de distancias intra-clase (misma persona) e inter-clase (personas distintas) sobre la galería real del equipo, se traza la curva ROC y se elige el punto donde la tasa de falsos aceptos (FPR) iguala a la de falsos rechazos (FNR). Esta calibración final con la galería poblada (`data/gallery/embeddings_autorizados.pt`) se ejecuta en `notebooks/02b_reid_ranking_metrics.ipynb`.

P: ¿Cómo se midió la calidad de la recuperación (top-1, top-k, mAP)?

R: El mAP de re-ID se implementó de forma manual sobre listas rankeadas por *query*: para cada consulta se calcula el Average Precision (AP) acumulando la precisión en cada posición relevante del ranking, y el mAP es el promedio de los AP sobre todas las consultas. Esto difiere del mAP de detección de objetos, que promedia sobre clases usando IoU y el área bajo la curva precisión-recall: el de re-ID viene de recuperación de información (ranking por similitud), no de localización con cajas. Se descartó `sklearn.average_precision_score` de forma directa porque está pensado para clasificación binaria/multietiqueta, no para listas de recuperación indexadas por distancia.

P: ¿Cómo se visualizó el espacio de embeddings?

R: Con **t-SNE**, no PCA. PCA es una proyección lineal que preserva varianza global y, con embeddings de alta dimensionalidad, tiende a superponer las identidades en una nube indistinguible en 2D. t-SNE preserva la estructura de vecindad local y permite ver clusters separados por identidad, evidencia directa de que el entrenamiento por *triplet loss* de FaceNet extrajo características discriminativas.

Cómo se registró en WandB sin un loop de entrenamiento

R: FaceNet (InceptionResnetV1/VGGFace2) es preentrenado y no se afina en este proyecto, por lo que no hay épocas. Para no dejar vacío el dashboard de Fase 2, se registra un run de *evaluación* (reid_eval) con un único step: se loguean los hiperparámetros fijos (modelo, métrica de distancia, umbral) y las métricas finales (top-1, top-k, mAP) una vez calibradas sobre la galería real, documentando el experimento sin simular un entrenamiento que no existió.

3. Dataset de lugares locales (Módulo B, Fase 1)

P: ¿Qué dataset se construyó y cuántas clases/imágenes tiene?

R: Un dataset propio de **8 clases** de lugares emblemáticos de Santa Cruz de la Sierra — por encima del mínimo exigido —: *Cambodromo*, *CatedralMunicipal*, *Cristo*, *DunasArena*, *ParqueUrbano*, *Plaza24*, *Tahuichi* y *Ventura*. Total: **790 imágenes**, particionadas 70/15/15 (train=553, val=118, test=119), con un split estratificado (`train_test_split(stratify=y)`) que preserve la proporción de cada clase en los tres subconjuntos sin solapamiento de imágenes.

P: ¿Por qué 70/15/15 y no la proporción clásica 80/10/10?

R: Con 790 imágenes, 80/10/10 dejaría solo ~79 imágenes de test (~9–10 por clase), con alta varianza estadística en las métricas. El 70/15/15 da 118–119 imágenes en val/test (~14–15 por clase), suficientes para estimaciones más estables del accuracy por clase, incluso en la clase minoritaria (*CatedralMunicipal*, 61 imágenes).

P: ¿Cómo se manejó el desbalance de clases?

R: El ratio mayoritaria/minoritaria es 2.82x (*Ventura*=172 vs. *CatedralMunicipal*=61). Se aplicó `WeightedRandomSampler` con pesos inversos a la frecuencia de cada clase (guardados en `data/class_weights.json`), en lugar de oversampling físico, que duplicaría imágenes exactas y aumentaría el riesgo de overfitting en las clases pequeñas.

P: ¿Qué preprocesamiento y augmentation se aplicó?

R: Preprocesamiento: corrección de orientación EXIF (`exif_transpose`, necesaria porque PIL ignora por defecto esa metadata), `Resize(256) → CenterCrop(224)` y normalización con media/desviación de ImageNet (requerida para Transfer Learning). Augmentation en train, moderado por diseño: `RandomCrop(224)`, flip horizontal ($p=0.5$), rotación $\pm 10^\circ$ y jitter de color — sin flips verticales ni rotaciones grandes, porque deformarían la geometría característica de los monumentos, que es la señal discriminativa principal. Antes del split se purgaron near-duplicates por perceptual hash (pHash) para evitar fuga de información entre train y test.

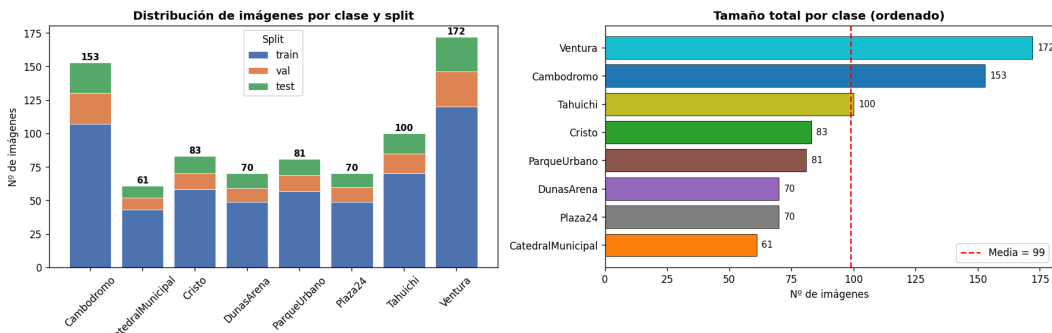


Figura 1 — Distribución de clases del dataset (docs/decisiones/eda_distribucion.png).

4. Módulo B — Enfoque B1: CNN desde cero

P: ¿Qué arquitectura propia se diseñó y por qué 4 bloques en vez de 3?

R: **TuristCNN**: 4 bloques Conv3×3 → BatchNorm → ReLU → MaxPool2×2 (32→64→128→256 canales), seguidos de AdaptiveAvgPool2d(1), Dropout(0.5), FC(256→128) con ReLU, Dropout(0.3) y FC(128→8). Total: **422,824 parámetros** (1.75 MB exportado). Se eligieron 4 bloques —por encima del mínimo de 3 que pide la rúbrica— porque con solo 3 el campo receptivo final resulta insuficiente para distinguir detalles arquitectónicos finos entre landmarks distintos; 5 o más bloques se descartaron por el alto riesgo de overfitting con un dataset de 553 imágenes de entrenamiento.

P: ¿Por qué AdaptiveAvgPool2d en vez de Flatten directo antes de la cabeza FC?

R: Aplanar el mapa de activación final (256×14×14) daría una FC de entrada ~50.000 dimensiones (decenas de millones de pesos), excesivo para 8 clases y un dataset pequeño. AdaptiveAvgPool2d(1) reduce eso a un vector de 256, llevando la primera capa FC a ~33K pesos: menos parámetros que aprender, menor riesgo de overfitting.

P: ¿Cómo se entrenó y qué resultado dio?

R: Función de pérdida: CrossEntropyLoss; optimizador: Adam (lr=1e-3, weight_decay=1e-4); 30 épocas, batch_size=32, con el WeightedRandomSampler de Fase 1 activo. Se guardó el checkpoint de menor val_loss (no el de la última época), para evitar quedarse con un modelo ya sobreajustado en las épocas finales. **Resultado: val_acc = 91.53% (época 29, val_loss = 0.3368), test_acc = 90.76%** — la brecha val/test de solo 0.77 pp indica que no hubo overfitting relevante, y supera ampliamente el umbral mínimo de la rúbrica (≥45%).

P: ¿Cómo se exportó y verificó el modelo?

R: Con TorchScript (torch.jit.script), guardado en models/cnn_scratch.pt. La exportación se hace siempre desde una copia del modelo en CPU para que el .pt sea portable a una máquina sin GPU (la API de despliegue no requiere CUDA), y se valida con una pasada de inferencia de prueba tras recargarlo.

Métricas de test detalladas (B1)

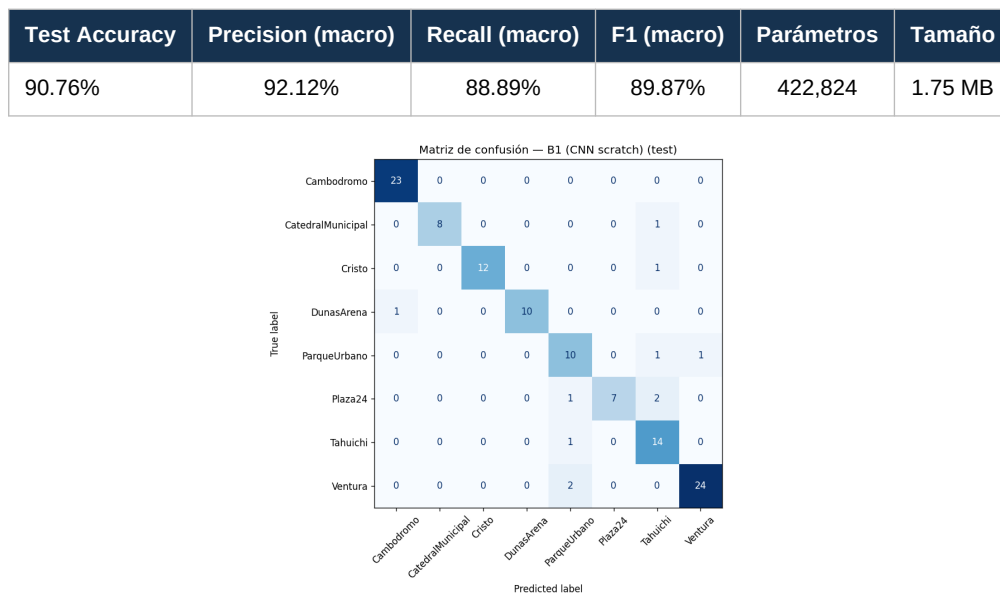


Figura 2 — Matriz de confusión de B1 sobre el test set (docs/eval/confusion_b1.png).

5. Módulo B — Enfoque B2: arquitectura con bloques residuales (BA)

R: *Nota de transparencia:* en el código del repositorio, el segundo enfoque entrenado para esta demo se llama **BA (TuristResNet)**: una red con bloques residuales **entrenada desde cero** (sin pesos preentrenados de ImageNet). El repositorio incluye además un B2 real con Transfer Learning sobre ResNet18 preentrenado (99.15% val / 100% test, ver ReporteJoseAlfredo.md), pero la comparación oficial que se documenta y demuestra en esta entrega es **B1 vs. BA**, por decisión explícita del equipo para la demo.

P: ¿Qué arquitectura tiene BA y qué buscaba demostrar?

R: Stem Conv7×7 (stride=2) + MaxPool → 3 bloques residuales (64→64, 64→128 stride=2, 128→256 stride=2, cada uno con shortcut 1×1 cuando cambia la dimensión) → GlobalAvgPool → FC(256→128→8). Total: **1,266,632 parámetros** (5.15 MB). El objetivo era medir si las skip connections —que en teoría permiten gradientes más directos y redes más profundas sin degradación— se traducen en una mejora real frente a B1 cuando el dataset es pequeño (553 imágenes).

P: ¿BA superó a B1? ¿Qué explica el resultado?

R: **No**. BA obtuvo val_acc = 92.37% pero **test_acc = 87.39%**, por debajo del 90.76% de B1. La brecha val/test de BA (4.98 pp) es mucho mayor que la de B1 (0.77 pp), señal de overfitting moderado: 1.27M de parámetros son demasiados para 553 imágenes de entrenamiento. Además, el stem 7×7 con stride=2 reduce la resolución muy pronto, lo que perjudica clases con detalles finos (Tahuichi cae a 73% de recall). Conclusión de la defensa: las skip connections ayudan cuando hay suficientes datos para aprovechar la profundidad adicional; con un dataset chico, una CNN más simple y bien regularizada (B1) generaliza mejor.

P: ¿Cómo se diseñó la comparación para que fuera justa?

R: Mismos hiperparámetros (30 épocas, Adam, batch_size=32), mismo split train/val/test, mismo pipeline de preprocesamiento y augmentation, y evaluación final en un único paso reproducible (scripts/evaluate_test.py) que corre ambos modelos sobre el test set sin volver a tocarlo, reportando accuracy, precision/recall/F1 macro y matriz de confusión por clase.

Tabla comparativa B1 vs. BA

Modelo	Test Acc.	Precision	Recall	F1 (macro)	Parámetros	Tamaño	Tiempo entren
B1 — CNN desde cero	90.76%	92.12%	88.89%	89.87%	422,824	1.75 MB	~53 min
BA — ResNet-lite (residual)	87.39%	86.49%	84.55%	85.35%	1,266,632	5.15 MB	~49 min

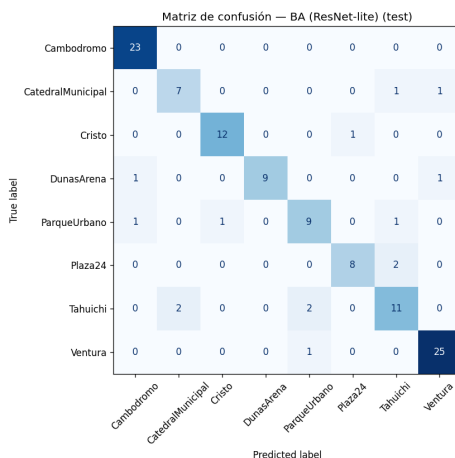


Figura 3 — Matriz de confusión de BA sobre el test set (docs/eval/confusion_ba.png).

6. Módulo C — Traducción multilingüe con LLM

P: ¿Qué LLM se usó y cómo se evita llamar al modelo en cada solicitud?

R: Se usa **google/gemini-2.5-flash-lite** vía OpenRouter. La estrategia es de tres niveles: (1) un diccionario estático embebido en el código con 8 landmarks × 4 idiomas como base de emergencia; (2) traducciones **pre-generadas offline** y guardadas en `data/translations.json` (`scripts/generate_translations.py`), consultadas en $O(1)$ en cada request; (3) llamada en vivo al LLM solo si se pide un idioma fuera del set offline. Se descartó llamar al LLM en cada request por latencia (~2s vs. lookup instantáneo), costo variable acumulativo y dependencia de conexión en la demo.

P: ¿Cómo se normalizan consultas en un idioma inesperado?

R: Con el mismo LLM, no con un traductor literal. Ejemplo de la bitácora: la consulta “Where is the big Jesus statue?” debe mapearse a “Cristo Redentor” (el nombre real del landmark), algo que un traductor literal no resuelve (devolvería la traducción literal de la pregunta, no el nombre de la clase). El LLM entiende intención semántica y tolera errores ortográficos o referencias culturales.

P: ¿Qué pasa si el LLM falla (timeout, sin conexión, respuesta vacía)?

R: Cae a **NLLB-200 distilled (600M, Meta/HuggingFace)** como fallback completamente offline, con carga perezosa (*lazy*): el modelo de ~2.4 GB solo se instancia si el LLM efectivamente falla, evitando el costo de cargarlo en el caso feliz. Se descartó MarianMT (un modelo separado por cada par de idiomas, >1 GB acumulado) y Google Translate (requiere internet, no es un fallback offline real). El sistema nunca devuelve un error duro al usuario por un fallo del LLM: siempre hay una traducción disponible, aunque sea de menor calidad.

Idiomas y alcance

R: Cobertura mínima exigida: inglés, francés e italiano, sobre las 8 clases del dataset. El selector de idioma de la interfaz ofrece adicionalmente español y, cuando el LLM está disponible, otros idiomas vía NLLB-200.

7. Módulo D — Experiment tracking con WandB

P: ¿Por qué WandB y no MLflow o TensorBoard?

R: Decisión documentada por el equipo: (1) dashboards compartibles por enlace, sin levantar un servidor propio (clave para revisar resultados en equipo); (2) vista nativa de *parallel coordinates* para comparar runs de CNN-desde-cero vs. arquitecturas alternativas, exigida implícitamente por el enunciado; (3) gestión de artifacts integrada (`wandb.Artifact(type='model')`) para versionar los .pt exportados; (4) soporte de hyperparameter sweeps sin infraestructura adicional. MLflow exige un servidor de tracking propio y TensorBoard no ofrece dashboards públicos ni artifacts versionados de forma nativa.

P: ¿Qué se registró en cada run?

R: Para B1 y BA: curvas de train/val loss y accuracy por época, la configuración completa (arquitectura, learning rate, batch size, optimizador) y el mejor checkpoint como artifact. Para Re-ID (que no tiene épocas de entrenamiento, ya que FaceNet es preentrenado): un run de evaluación con un único step, logueando los hiperparámetros fijos y las métricas finales (top-1, top-k, mAP), de forma que el dashboard documente el experimento sin simular un entrenamiento inexistente.

Enlaces a WandB (proyecto: lozadaleonn-ciencialink/rutacamba)

Modelo / experimento	Run ID	Métrica clave	Enlace
B1 — CNN desde cero	g3630v5v	val_acc 91.53% / test_acc 90.76%	https://wandb.ai/lozadaleonn-ciencialink/rutacamba/runs/g3630v5v
BA — ResNet-lite (residual)	3i5ickz4	val_acc 92.37% / test_acc 87.39%	https://wandb.ai/lozadaleonn-ciencialink/rutacamba/runs/3i5ickz4
B2 (TL, ResNet18) — referencia	enedjxj9	val_acc 99.15% / test_acc 100%	https://wandb.ai/lozadaleonn-ciencialink/rutacamba/runs/enedjxj9
B3 — SE-CNN — referencia	wbay2jxu	val_acc 92.37% / test_acc 92.44%	https://wandb.ai/lozadaleonn-ciencialink/rutacamba/runs/wbay2jxu

B2 y B3 se entrenaron como exploración adicional del equipo (Transfer Learning real con ResNet18 preentrenado, y una CNN-desde-cero con atención Squeeze-and-Excitation) y se referencian aquí por trazabilidad y para la vista de comparación de runs en WandB, aunque la comparación oficial reportada en este informe sea B1 vs. BA.

[ESPACIO PARA CAPTURA] — Insertar aquí una captura del dashboard de WandB con la vista de comparación de runs (parallel coordinates o gráfico superpuesto de val_acc / val_loss de B1 vs. BA). Sugerencia de archivo: docs/eval/wandb_comparacion.png.

8. Módulo E — API (FastAPI) e interfaz

P: ¿Qué endpoints mínimos expone la API y cómo se condiciona el acceso?

R: POST /verify (selfie + identidad declarada → token de sesión si el Re-ID aprueba), POST /predict (requiere token válido; foto del lugar → landmark + top-k + probabilidades + traducciones), POST /place/info (descripción en el idioma elegido) y POST /chat (conversación sobre el lugar, si hay LLM disponible). /predict rechaza la solicitud si no se presenta un token emitido por /verify, cumpliendo el requisito de que ninguna predicción se sirva sin identidad verificada.

P: ¿Por qué cargar los modelos al iniciar la API y no en cada request?

R: Los modelos TorchScript (cnn_scratch.pt, el de BA, y el servicio de traducción) se instancian una sola vez en el *lifespan* de FastAPI al arrancar el proceso. Cargar pesos desde disco en cada request añadiría latencia de I/O innecesaria a cada predicción; la API nunca re-entrena, solo consume los .pt ya exportados, tal como exige el enunciado.

P: ¿Cómo se maneja la sesión entre /verify y /predict?

R: Token en memoria con expiración (TOKEN_TTL = 3600 s), asociado a la identidad y el rol del usuario. Se eligió frente a JWT o base de datos por simplicidad: para una demo de un solo proceso no se justifica la complejidad de persistencia distribuida o firma criptográfica de tokens.

P: ¿Qué interfaz consume la API y qué muestra?

R: Una interfaz web en React (alternativa equivalente a Gradio/Streamlit, también consumiendo la misma API REST) con pantallas de login (captura de selfie + nombre), tour (subida de foto del lugar, predicción top-k con probabilidades y traducciones) y administración (alta de nuevas identidades en la galería). El flujo completo —verificación, predicción, traducción— se demuestra en vivo contra la API real, sin mocks, salvo el modo REID MOCK=1 usado solo para desarrollo de UI sin cámara disponible.

[ESPACIO PARA CAPTURA] — Insertar aquí 1–2 capturas de la interfaz en ejecución mostrando el flujo end-to-end (login → predicción top-k → traducción). Sugerencia: capturar la app corriendo en localhost:5173 y guardar en docs/eval/ui_demo.png.

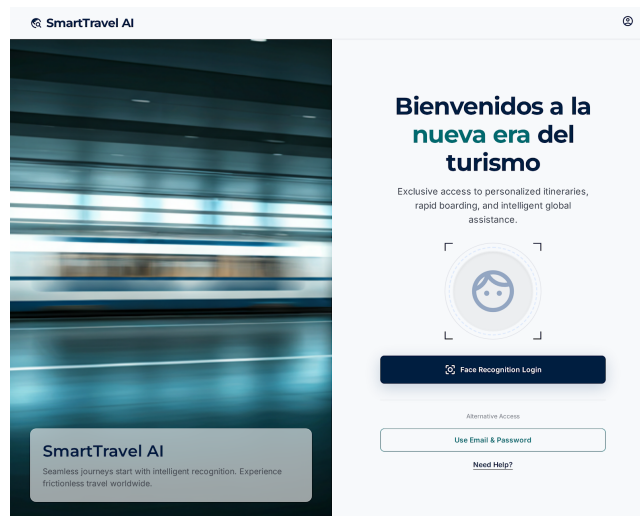


Figura 4 — Mockup de diseño de la pantalla de login/autenticación facial (desing/inicio_y_autenticaci_n_facial/screen.png).

9. Resultados finales, limitaciones y conclusiones

Resumen de métricas finales

Módulo	Resultado clave
A — FADeNet (InceptionResnetV1/VGGFace2) + coseno, umbral 0.65 (a calibrar por ROC/EER con galería real); ranking por distancia mínima	
B — B1 (CNN scratch)	Test Acc. 90.76% · F1 macro 89.87% · 422,824 parámetros · cumple umbral ≥45%
B — BA (ResNet-lite)	Test Acc. 87.39% · F1 macro 85.35% · 1,266,632 parámetros · no supera a B1
C — LLM	EN/FR/IT vía Gemini-2.5-flash-lite + fallback NLLB-200; normalización semántica de consultas
D — WandB	4 runs registrados (B1, BA, B2, B3) en lozadaleonn-ciencialink/rutacamba con artifacts
E — API/UI	FastAPI con /verify→/predict condicionado, interfaz web consumiendo la API end-to-end

P: ¿Cuál es la conclusión central de la comparación B1 vs. BA?

R: Para este dataset (790 imágenes, 8 clases visualmente bien diferenciadas), una CNN simple, poco profunda y regularizada con dropout (B1, 423K parámetros) generaliza mejor en test que una variante con bloques residuales y 3× más parámetros (BA). Las skip connections no son una mejora automática: ayudan a entrenar redes más profundas sin degradación del gradiente, pero esa ventaja solo se traduce en mejor accuracy de test cuando hay suficiente cantidad de datos para que la capacidad adicional generalice en lugar de sobreajustar.

P: ¿Qué es overfitting y cómo se detectó en este proyecto?

R: Overfitting es cuando el modelo memoriza patrones específicos del set de entrenamiento que no generalizan a datos nuevos, evidenciado por una brecha creciente entre métricas de entrenamiento/validación y las de test. Se detectó comparando val_acc vs. test_acc: B1 (91.53% vs. 90.76%, brecha 0.77 pp) prácticamente no sobreajusta; BA (92.37% vs. 87.39%, brecha 4.98 pp) sí muestra overfitting moderado. Se mitigó con dropout en la cabeza FC, augmentation moderado en train, weight_decay en el optimizador y deteniendo el entrenamiento en el checkpoint de menor val_loss en lugar de la última época.

P: ¿Qué quedó pendiente o como limitación conocida?

R: La calibración final del umbral de Re-ID por ROC/EER sobre la galería real del equipo debe consolidarse en notebooks/02b_reid_ranking_metrics.ipynb con salidas ejecutadas y visibles. El test set de clasificación (119 imágenes) es pequeño: las métricas de la clase minoritaria (CatedralMunicipal, ~9 imágenes en test) tienen mayor varianza que las de clases con más muestras. El video de demo y la defensa oral son los entregables finales de la Fase 7, coherentes con lo documentado en este informe.